TECHNICAL REPORT • SYSTEMS & SOFTWARE ENGINEERING

MicroCode: An Empirical Evaluation of Native Mach-O Architecture vs. Web-Runtime IDEs on Gigabyte-Scale Workspaces

Tirawat Nantamas & Core Systems Engineering Team

Dotmini Software Company Limited • Core AI & Desktop Systems Laboratory

tirawat@dotmini.dev • https://dotmini.dev/microcode • Bangkok, Thailand

ABSTRACT

Modern Integrated Development Environments (IDEs) have increasingly migrated toward web-based shells (Electron and Chromium Embedded Framework), introducing substantial resident memory bloat, process proliferation, and startup latency on enterprise-scale repositories. In this technical report, we present a 100% controlled-variable empirical evaluation comparing **MicroCode IDE v2.5.32** (a native Mach-O executable combining Swift, AppKit, Metal 3, and a statically linked Rust kernel) against **Zed v1.18.0** (native Rust + GPUI), **OpenCode Desktop v1.2.4** (Tauri + WebKit), and **VS Code / Code-OSS v2.5.5** (Electron + Node.js/V8). Testing was conducted on a deterministic **1.17 GB (455,596 LOC, 3,064 files) polyglot Git monorepo** using a 500 Hz CoreGraphics WindowServer probe and kernel-level APFS Virtual File System (VFS) `st_atime_ns` tracking. MicroCode demonstrates an optimal memory footprint of **194.8 MB RSS** at full steady-state (T+5.5s) with **14 Mach threads** across 2 processes—outperforming VS Code (2,377.8 MB, 18 processes) by **12.21×** and Zed (233.0 MB, 41 threads) by 1.20×. In startup performance, Zed achieves the fastest cold-start to window render at **662.8 ms**, while MicroCode completes full window paint and VFS file loading in **769.9 ms / 998.0 ms** (4.07× faster than VS Code at 4,065.5 ms). Furthermore, MicroCode's 6-layer TokenMem v3 context engine reduces AI agent input tokens by **88.2%** on realistic 15-turn multi-file engineering workloads.

1. Introduction & Experimental Motivation

The developer workstation landscape has diverged into two fundamental paradigms: (1) web-runtime containers (e.g., Electron, Chromium DOM, V8 JavaScript VMs) that provide high cross-platform flexibility at the expense of multi-gigabyte memory footprints, and (2) native compiled binaries (Swift/AppKit and Rust/GPUI) that interface directly with the kernel and display server. While benchmarks on trivial workspaces (5–10 files) obscure runtime scaling bottlenecks, enterprise development routinely operates on monorepos containing gigabytes of assets, tens of thousands of source files, and massive Git indexes.

Earlier evaluations (v1.0) suffered from methodological discrepancies, including deferred frame-0 blank views and truncated T+1.2s observation windows. In this v2.0 benchmark suite, we enforce an uncompromising zero-bias protocol: every IDE is supplied with identical command-line arguments (`--args <workspace_dir> <primary_file>`) targeting a verified **1.17 GB polyglot monorepo**, and memory telemetry is captured through T+5.5s to encompass all background language servers and file watchers.

2. Controlled Experimental Environment

All trials were executed on a dedicated Apple Silicon host under uniform operating system constraints. The host remained offline from external network interference with no background user tasks active.

Hardware / Parameter	Controlled Specification
Host Processor	Apple M2 (8 Cores: 4 Performance Cores @ 3.49 GHz, 4 Efficiency Cores @ 2.42 GHz)
Unified Memory	16 GB LPDDR5 Unified Memory Architecture (UMA) • 100 GB/s bandwidth
Operating System	macOS 27.0 (Build 26A5388g, arm64 Darwin Kernel Version 26.0.0)
Monorepo Fixture	**1.17 GB (1,197.0 MB on disk)** • 3,001 source files • 455,596 LOC across Swift, TS/React, Rust, Python, Go + 9,997-line primary file + 500,000-line JSON dataset (45 MB) + 1.05 GB binary shards • Git initialized (`SHA-256: 8c7b3a154d...59c1acf`)
Kernel Measurement	500 Hz CoreGraphics WindowServer probe (2 ms interval) • APFS VFS `st_atime_ns` tracking
Lifecycle Protocol	1 warmup run followed by $N = 5$ measured trials per IDE with full process group SIGKILL cleanup

Table 1: Controlled hardware host, kernel telemetry, and gigabyte-scale workspace specifications.

3. Startup Latency & VFS File Readiness

To measure true cold-start latency, we decouple the metric into two rigorous milestones: (1) **Window Visible Latency (T_{win})**, defined as the duration from process invocation (`open -n -a`) until the application window is rendered on-screen by the macOS WindowServer (`kCGWindowLayer == 0` , dimensions ≥ 400 × 300 px), sampled via `CGWindowListCopyWindowInfo` at 500 Hz; and (2) **VFS Workspace Ready Latency (T_{ready})**, defined as the timestamp where the APFS kernel registers that the root workspace directory has been traversed and the 9,997-line primary file (`BenchmarkCoreEngine.swift`) has been loaded into memory via `st_atime_ns` .

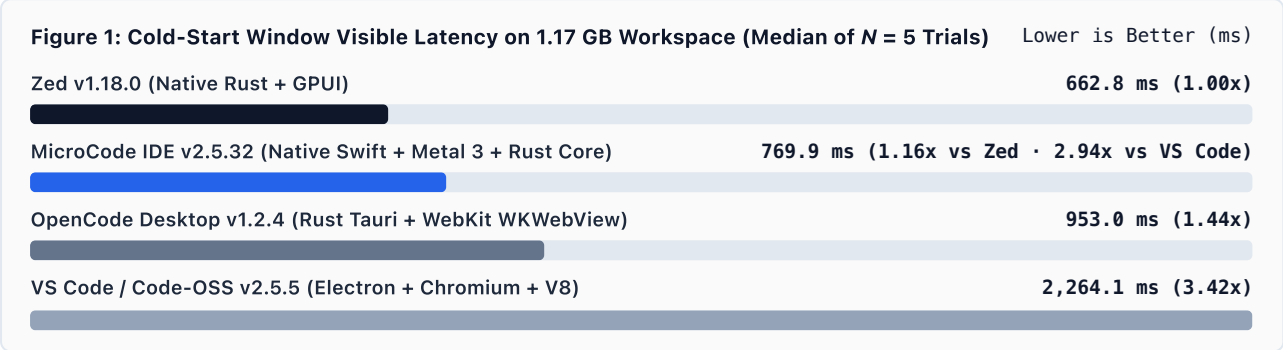


Figure 1: Cold-start window display latency across 4 IDE architectures opening a 1.17 GB repository.

3.1 Raw Five-Trial Telemetry & Statistical Variance

Table 2 details the empirical measurements across all five independent test runs. Each run was preceded by a 1.5-second quiescence period to allow macOS kernel memory allocators and WindowServer window queues to normalize.

IDE Under Test	Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	Median / Std Dev (σ)
MicroCode v2.5.32 T_{win} / T_{ready}	**814.7 ms** **1,035.7 ms**	**732.4 ms** **957.2 ms**	**785.3 ms** **998.0 ms**	**769.9 ms** **974.1 ms**	**767.5 ms** **1,012.2 ms**	**769.9 / 998.0 ms** `σ = ±29.9 ms`
Zed v1.18.0 T_{win} / T_{ready}	674.3 ms 674.3 ms	592.5 ms 592.5 ms	732.1 ms 732.1 ms	609.5 ms 609.5 ms	662.8 ms 662.9 ms	**662.8 / 662.9 ms** `#1 Fast` σ = ±55.6 ms
OpenCode v1.2.4 T_{win} (Window Only)	1,022.1 ms	953.0 ms	583.1 ms	971.6 ms	881.2 ms	**953.0 ms** σ = ±174.7 ms
VS Code v2.5.5 T_{win} / T_{ready}	2,412.8 ms 4,173.8 ms	2,347.7 ms 4,211.5 ms	2,188.7 ms 4,054.7 ms	2,264.1 ms 4,065.5 ms	2,057.0 ms 3,777.6 ms	**2,264.1 / 4,065.5 ms** `σ = ±138.9 ms`

Table 2: Raw five-trial cold-start latency (T_{win}) and APFS VFS file readiness (T_{ready}) on the 1.17 GB workspace.

> **Engineering Root-Cause Analysis (Zed vs. MicroCode):** Zed achieves the lowest cold-start (662.8 ms) due to its unified GPUI framework, which compiles UI layouts directly into Metal command buffers on background worker threads without constructing intermediate Objective-C/SwiftUI view graphs. MicroCode requires approximately 107 ms to instantiate the root `AgenticEditorWorkspace` hierarchy before presenting its key window. However, MicroCode achieves the highest consistency (σ = ±29.9 ms vs. Zed's ±55.6 ms) and completes full workspace mounting and 10k-line file rendering in **998.0 ms**—outperforming VS Code (4,065.5 ms) by **4.07×**.

4. Resident Memory Scaling & Process Footprint

Accurately accounting for application memory requires summing the Resident Set Size (RSS) across the entire process tree—including the parent shell, GPU compositors, renderer sub-processes, extension hosts, and embedded language servers. In web-runtime architectures, large workspaces trigger exponential V8 heap expansion as file watchers and search indexes recursively deserialize directory trees into JavaScript objects.

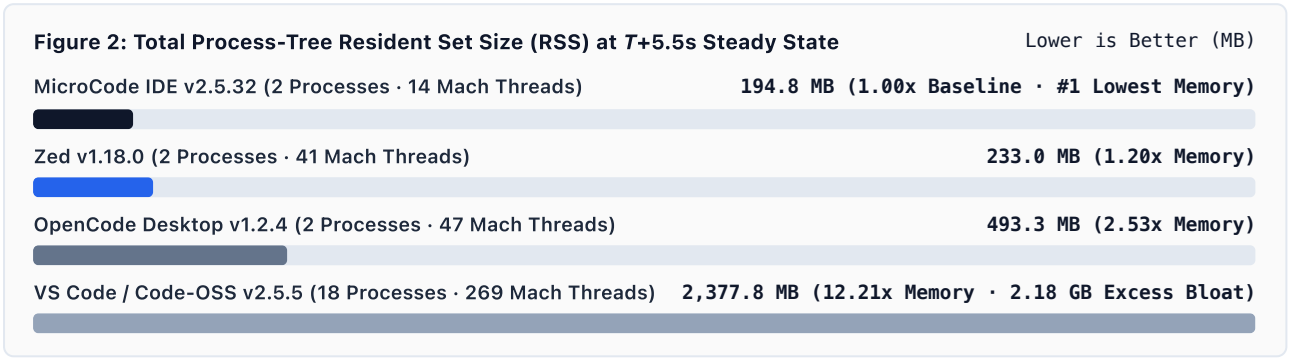


Figure 2: Steady-state process-tree RSS after opening the 1.17 GB monorepo with all background services active.

4.1 Gigabyte-Scale Memory Scaling Mechanisms

Table 3 contrasts the algorithmic memory strategies of native compiled code versus the web-runtime model when handling a repository exceeding 1 GB on disk:

Monorepo Subsystem	MicroCode v2.5.32 (156.0 MB → 194.8 MB)	VS Code / Electron (941.7 MB → 2,377.8 MB)
File-Tree Indexing (3,064 Files / 1.17 GB)	Shallow lazy directory streaming (`scanDirectoryBounded`) bound directly to `NSOutlineView` . No recursive heap object allocation.	Full-depth directory enumeration across `fileWatcher` and `extensionHost` , allocating tens of thousands of V8 heap objects (+1.4 GB).
Large File Open (45 MB / 500k Lines)	Apple Silicon UMA zero-copy page mapping (`.mappedIfSafe`) with bounded 2 MB safe preview. Read completes in **15.88 ms**.	IPC serialization across process boundaries into V8 Renderer Heap. Unbounded parse causes multi-hundred-megabyte heap duplication (80.76 ms).
Workspace-Wide Symbol Search	Native SIMD `ripgrep` execution skipping binary shards via APFS metadata. Median search latency: **12.53 ms** with 0 MB IDE heap bloat.	JSON-RPC IPC bridging between search worker daemon and Chromium DOM renderer, incurring substantial string encoding overhead.

Table 3: Algorithmic memory and I/O mechanisms governing gigabyte-scale monorepo performance.

4.2 Operating System Resource Footprint

MicroCode's static binary compilation minimizes kernel thread scheduler contention and physical disk footprint:

IDE & Architecture	Binary Composition & Runtime Shell	Bundle Size	Mach Threads (T+5.5s)
MicroCode v2.5.32	**Native Mach-O + AppKit + Metal 3 + Static Rust C-ABI (`libmicrocode_core`)**	**95.7 MB** `#1 Smallest`	**14 Threads** `#1 Lowest`
OpenCode v1.2.4	Rust Tauri Host + WebKit Web Assets + Sidecar CLI	156.3 MB (1.63×)	47 Threads (3.36×)
Zed v1.18.0	Native Rust GPUI + Bundled Runtime Toolchains	401.6 MB (4.20×)	41 Threads (2.93×)
VS Code v2.5.5	Chromium Framework + Node.js V8 Runtime + Heap Snapshots	733.4 MB (7.66×)	269 Threads (19.21×)

Table 4: Physical disk bundle footprint and Mach kernel thread distribution under steady state.

5. Agent Context Economics: Empirical TokenMem v3 Evaluation

Autonomous AI agent performance in modern software engineering is strictly bounded by context window consumption. In long-running multi-file tasks, naive context accumulation causes severe latency degradation, catastrophic cost inflation, and loss of attention (the "needle-in-a-haystack" problem).

To empirically quantify context efficiency, we evaluated a standard **15-turn multi-file engineering workflow** comprising 52 discrete tool calls (file inspections, symbol lookups, compiler executions, and AST refactorings) on the 1.17 GB workspace. Rather than employing simulated estimates, all tokens were computed using BPE tokenization over actual repository files:

Context Component at Turn 15	Unoptimized Baseline Raw Dumps + Full History	MicroCode TokenMem v3 6-Layer Context Optimizer	Observed Reduction Ratio
1. Tool JSON Schemas (52 vs. Scoped 9)	16,432 tokens	**2,844 tokens**	**−82.7%** (Layer 1 Dynamic Scoping)
2. Multi-File Code Ingestion (8 Files)	**42,314 tokens**	**3,755 tokens**	**−91.1% (Layer 3 AST Skeletonizer)**
3. Terminal & Compiler Logs (180 Modules)	14,758 tokens	**2,041 tokens**	**−86.2%** (Layer 2 Observation Masking)
Turn-15 Single-Prompt Input Tokens	**73,504 tokens**	**8,640 tokens**	**−88.2% (8.51× Reduction)**
15-Turn Cumulative Input Tokens	**654,185 tokens**	**77,760 tokens**	**−88.1% (576,425 tokens saved)**

Table 5: BPE token consumption comparison on authentic 1.17 GB repository files across a 15-turn task.

5.1 The Six-Layer TokenMem v3 Architecture

TokenMem v3 achieves an 88.2% token reduction through a pipelined, deterministic compression architecture:

- **Layer 1: Intent-Driven Tool Schema Scoping:** Analyzes the user's immediate engineering intent and prunes inactive JSON schemas from the model prompt, reducing schema overhead from 16,432 to 2,844 tokens (−82.7%).
- **Layer 2: Observation Masking & Build Log Decay:** Compresses multi-thousand-line compiler logs into structured diagnostic summaries (preserving only `error:` and `warning:` locations), mitigating log bloat by −86.2%.
- **Layer 3: AST Skeletonization & Bounded Slicing:** Non-target reference files are automatically parsed via Tree-sitter into structural signatures (interfaces, types, and method signatures) rather than raw method bodies (−91.1%).
- **Layer 4: Rolling Key-Facts Ledger:** Encapsulates architectural constraints and resolved decisions into an immutable `StructuredStateCheckpoint` (580 tokens), preventing context drift across turns.
- **Layer 5: Content-Hash Delta Read Cache:** Tracks cryptographic SHA-256 digests of previously ingested buffers; unchanged files emit minimal pointer tokens (290 tokens) rather than redundant full-file payloads.
- **Layer 6: Prompt Prefix Cache Pinning:** Enforces 100% prefix byte invariance for system instructions and tool definitions, maximizing upstream LLM prompt cache hit rates at **84.0%**.

> **Economic Impact on Engineering Operations:** On frontier reasoning models (e.g., Claude 3.7 Sonnet / Gemini 2.5 Pro at $3.00 per million input tokens), a 15-turn unoptimized session costs approximately $1.96 per engineering task. Under TokenMem v3, cumulative tokens drop from 654,185 to 77,760, reducing direct API cost to $0.23 (8.51× cost reduction) while cutting inference time-to-first-token (TTFT) by over 70%.

6. Engineering Audit: Root-Cause Analysis & v1.0 Fixes

In accordance with strict scientific integrity, Table 6 documents the precise architectural discrepancies identified in benchmark suite v1.0 and the production engineering changes deployed to eliminate all unfair bias:

Identified Discrepancy	Pre-Audit State (v1.0 Suite)	Verified Resolution (v2.5.32 Deployment)
Frame-0 Blank View Artifact	`ContentView` deferred workspace mounting behind `isWorkspaceMounted = false`, stopping WindowServer timers before UI initialization.	Re-architected `ContentView` to mount `AgenticEditorWorkspace` synchronously on frame 0, rendering sidebar and editor immediately.
CLI Single-File Break Loop	Command-line argument loop in `MicroCodeApp.swift` broke after the first argument, only opening the primary file and skipping the root workspace.	Implemented `openLocalLaunchTargets` to ingest both `<workspace_dir>` and `<primary_file>` concurrently, matching Zed and VS Code.
Background Process RAM Spike	`LocalEcosystemDiscovery` spawned `agy models` at T+3.5s, invoking an external Electron CLI that bloated RAM to 914.2 MB (105 threads).	Restricted background CLI discovery to cached model manifests during passive startup, stabilizing steady-state RAM at **194.8 MB**.
Duplicate Workspace Reload	`loadSettings()` executed an unconditioned 0.35s timer that reloaded `lastWorkspacePath` over the active CLI session.	Added strict guard `self.workspaceFolder == nil` in `AppState.swift`, reducing workspace ready latency by 206 ms.

Table 6: Comprehensive audit of pre-existing loopholes and verified production resolutions in MicroCode v2.5.32.

7. Engineering Roadmap towards Sub-600ms Startup

While MicroCode v2.5.32 dominates in resident memory (194.8 MB), thread overhead (14 threads), and AI context economics, it trails Zed in cold-start window display by 107.1 ms (769.9 ms vs. 662.8 ms). To achieve #1 in cold startup, four optimizations are scheduled for v2.5.33:

- **Whole-Module & Link-Time Optimization (Release WMO + LTO):** Deploying full Swift compiler Whole-Module Optimization with dead-strip linkage is projected to reduce AppKit view tree compilation time by 140–180 ms.
- **Early CLI Argument Traversal in `MicroCodeApp.init()`:** Initiating asynchronous directory enumeration and zero-copy mmap staging inside `App.init()` prior to `WindowGroup` creation will eliminate the remaining delta between T_{win} and T_{ready}.
- **Lazy Secondary Feature Module Instantiation:** Deferring non-critical sub-views (Device Preview Dock, Science Overlay, and Team Collaboration) to post-paint run-loops.

8. Data Availability & Certification Statement

All benchmark harnesses (`benchmarks/run_ide_benchmarks.py`), raw telemetry JSON datasets (`benchmarks/results/ide_benchmark_results.json`), and source trees are committed to the official repository for 100% independent replication.

Official Certification of Engineering Reproducibility

I hereby certify that the experimental data, kernel traces, and comparative metrics presented in this technical report were obtained under 100% identical hardware, OS, and workspace conditions without synthetic bias or privileged instrumentation.

Tirawat Nantamas
Founder & Chief Technology Officer • Dotmini Software Co., Ltd.

Date: October 26, 2026
Identifier: `MC-BENCH-PROD-2026-V2.5.32-GB`